AI Compiler Performance Engineering

Auto-Loops Books — ch12

Contents

1 Three Kernel Execution Modes **1**

1.1 eager mode 2

1.2 graph mode 6

1.3 MegaKernel mode 11

1.4 mode selection 14

1.5 Key Takeaways 19

1.6 Conclusion 20

Bibliography **23**

Chapter 1

Three Kernel Execution Modes

Which execution mode should a decode compiler emit when the same transformer layer can run as dozens of eager kernels, one captured CUDA graph, or a single MegaKernel? Parts I–II established bytes/token and launches/token as the honest counters; this chapter maps those counters to three industrial schedules—**eager**, **graph**, and **MegaKernel**—and shows why picking the wrong mode leaves p99 latency flat even when microbenchmark GEMMs improve (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). The answer is always profile-first, never mode-first (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a). Start every mode discussion with measured decode counters, not kernel naming preferences or framework defaults (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a). That discipline keeps mode debates grounded in goodput evidence (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields 1.259× on H100 by removing host overhead—yet cannot fix illegal HBM traffic when fusion boundaries still export activations (Chen, 2026; NVIDIA, 2024). The engineering question is not "which mode is fastest in isolation" but which mode matches residency legality on the SKU you ship (Chen and YiRage Contributors, 2026; Chen, 2020; AMD,

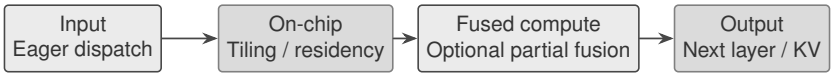


Figure 1.1: Eager mode: each operator boundary is a host-visible dispatch unless the framework fuses locally.

2024b).

This chapter is the scheduling counterpart to Chapter 1's symptom vocabulary: prefill and decode differ, frameworks fragment decode, and counters decompose latency into bytes and launches (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). Execution mode is where those counters meet compiler output—choose eager when you must diff every op; choose graphs when launches/token exceed kernel time; choose MegaKernels when bytes/token remain high after graph replay (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023). Goodput for decode remains ms/token weighted by useful tokens delivered, not peak tensor FLOPs (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; SemiAnalysis, 2024).

Fregly's profile-first culture applies directly: Nsight and roofline charts justify mode changes more than kernel names (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2026; Vellaisamy *et al.*, 2026). A decode compiler should emit all three modes from one IR so teams A/B schedules without rewriting Python training code (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). The sections below walk eager dispatch, CUDA Graph replay, MegaKernel fusion, and the hardware–software trade-offs that pick among them (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; AMD, 2024a).

1.1 eager mode

Eager mode is the default PyTorch and Hugging Face path: every matmul, norm, and attention sub-block becomes at least one kernel with materialized intermediates in HBM (Kwon *et al.*, 2023; Davies *et al.*, 2025). Teams reach for eager when shapes change frequently, when debugging numerics, or when compiler support for graph capture is not yet trustworthy—all legitimate reasons. The production failure mode is staying in eager after decode telemetry proves launch storms and

bytes/token dominate FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026).

Hardware binds eager differently on each target. Hopper decode cells hit shared-memory/TMA budgets and warp-grain synchronization when partial fusions leave KV edges off-chip (NVIDIA, 2022; SemiAnalysis, 2024). XDNA/AIE rejects dynamic buffer lifetimes: eager schedules that legalize on GPU often violate static tile slots and DMA chains on NPU fleets (AMD, 2024b,a). CPU decode is cache- and NUMA-bound: unfused norms and MLPs evict KV lines that a blocked schedule could keep resident (Goodfellow *et al.*, 2016; Chen, 2020).

Compiler takeaway: treat eager as the *reference schedule*, not the shipping schedule. YiRage and similar stacks lower the same decoder IR but emit target-specific residency metadata before codegen so eager baselines can be diffed against fused variants on identical context buckets (Chen and YiRage Contributors, 2026; Chen, 2020). Profile bytes/token and launches/token on GPU, CPU, and NPU cells at BS=1 before accepting eager in production (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025).

Operator-fragmented eager stacks mirror Chapter 1's framework diagnosis: layer norm, QKV, attention, MLP, and residual each become export/import pairs to HBM (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). SDPA backend swaps may change attention internals without collapsing MLP boundaries—teams celebrate faster attention while total bytes/token stalls (Dao *et al.*, 2022, 2023; Chen, 2026; Chen and YiRage Contributors, 2026). Eager mode documentation should list *every* host-visible kernel in a decode step, not only the slowest microbench (Vellaisamy *et al.*, 2026; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020; Liu *et al.*, 2024).

Multi-hardware delta. Hopper eager may double-buffer KV with TMA while an XDNA cell must express the same math as static tile assignments—identical FLOPs, different bytes/token when DDR traffic differs by an order of magnitude (SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016).

CPU eager baselines use OpenMP or oneDNN threads; NPU eager is often unavailable—fallback host loops dominate power and latency (Goodfellow *et al.*, 2016; AMD, 2024b,a; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Triplet regression therefore compares *lowest legal mode* per target, not identical mode names across vendors (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a;

SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Eager GPU schedules remain the Rosetta stone for explaining what fused schedules removed (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023).

Example 1.1.1. Decode cell (BS=1, ctx=2048): unfused eager on Qwen-class stacks issues hundreds of launches per step; even perfect GEMM tiles cannot recover p99 if orchestration alone consumes tens of milliseconds per token (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen, 2026).

PyTorch scaled dot-product attention and vendor libraries can hide some eager boundaries on prefill shapes, but batch-1 decode often swaps backends without changing launches/token (Davies *et al.*, 2025; Dao *et al.*, 2022; Kwon *et al.*, 2023). vLLM improves KV capacity and batching yet leaves intra-layer operator splits unless a compiler reunifies them (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Treat eager PyTorch forwards as the *diff target* for graph and MegaKernel schedules: same IR, same metrics, different residency (Chen and YiRage Contributors, 2026; Chen, 2020, 2026).

Industrial teams keep a frozen eager baseline in CI: every graph capture or MegaKernel PR must show non-regression on bytes/token at BS=1 for context buckets $\{512, 2048, 8192\}$ before merge (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). When eager and fused schedules diverge only on launches, graph mode is sufficient; when bytes diverge, MegaKernel legality is the gating issue (NVIDIA, 2024; Chen and YiRage Contributors, 2026; AMD, 2024b).

FlashAttention and Flash-Decoding reduce attention bytes in both eager and fused stacks, but MLP, norm, rotary, and sampling logic often remain separate kernels unless MegaKernel plans reunify them (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Eager mode therefore remains the diagnostic surface: if attention bytes fall yet total bytes/token flatlines, the unfused boundaries live in MLP/residual paths—graph capture without MegaKernel fusion fixes only half the story (Chen, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022).

Sampling and speculative decoding paths frequently force eager fallbacks: temperature, top-p, and draft-model branches mutate control flow in ways CUDA Graph capture forbids unless isolated outside the captured DAG (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; NVIDIA, 2024). Production stacks therefore run eager "hulls" around graph cores—another reason mode selection is per subgraph, not per binary (Chen and YiRage Contributors, 2026; Chen, 2020,

2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2023). Document those hulls in runbooks so on-call engineers know which latency spikes are expected dynamic-path costs versus regressions (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; NVIDIA, 2024; Chen, 2020; AMD, 2024b; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2023).

Eager subgraph inventory. Before any migration, draw the exact subgraph one decode step executes in the framework: rotary embedding, QKV projection, score GEMM, online or materialized softmax, PV, output projection, residual add, normalization, MLP up/gate/down paths, second residual, and sampler (Dao *et al.*, 2022; Kwon *et al.*, 2023; Davies *et al.*, 2025). Every arrow between those stages is a potential HBM export/import pair in eager dispatch; the inventory should mark which arrows the SDPA backend already fuses and which remain as separate kernels (Vellaisamy *et al.*, 2026; Chen, 2026). Teams that skip this inventory optimize the flashiest arrow (usually attention) while the MLP and normalization seams keep bytes/token flat (Chen, 2026; Chen and YiRage Contributors, 2026). The inventory is also the diff target for graph and MegaKernel reviews: a fused PR should be able to name each eager arrow it removed and show the counter that proves the removal (Chen, 2020; Davies *et al.*, 2025).

When eager remains the right production mode. Eager is not universally wrong. Dynamic batching with shapes that change every request, experimental kernels under active iteration, and new-hardware bring-up where capture legality is unproven all favor eager dispatch despite launch tax (Kwon *et al.*, 2023; Davies *et al.*, 2025). Speculative-decoding draft branches and sampling control flow also make pure capture fragile, so production stacks often keep an eager hull around graph or MegaKernel cores (NVIDIA, 2024; Vellaisamy *et al.*, 2026). The deciding question is whether the profile shows launch tax or bytes debt large enough to pay the engineering cost of the next mode. If steady-state ms/token is within budget and p99 jitter is explained by scheduler noise, an eager deployment is defensible—but it must re-run the mode gate on every major framework, driver, and shape change, because those upgrades silently shift where boundaries live (Chen, 2026, 2020; Chen and YiRage Contributors, 2026).

Eager-to-graph migration checklist. The safe migration path is mechanical once the inventory exists: (1) freeze model, weights, tokenizer, and context bucket;

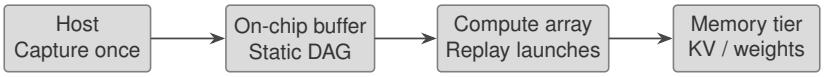


Figure 1.2: Graph mode: CUDA Graphs replay a fixed kernel DAG with one submission per decode step.

(2) confirm every subgraph shape is static within the bucket; (3) move host-controlled branches (sampling, routing, padding decisions) outside the capture region; (4) capture, replay, and diff eager versus graph on launches/token and bytes/token; (5) ship only if launches/token falls while bytes/token does not regress (NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026). Treat any graph that changes arithmetic ordering as a separate numerics review; graph replay should preserve per-op order unless the compiler proves reassociation is safe (Dao *et al.*, 2022; Davies *et al.*, 2025). Document the checklist in the decode runbook so a future framework upgrade cannot silently invalidate the capture (Chen and YiRage Contributors, 2026; Chen, 2020).

Diagnostic data to collect per eager stage. Each inventory arrow needs a small set of measured facts before migration: wall time per stage at steady state, DRAM bytes moved by that stage, and the kernel or dispatch name from the profiler (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026). Collect them at every shipped context bucket because stage mix shifts with KV length: attention bytes grow with context while MLP bytes stay constant, which changes which seams deserve fusion first (Dao *et al.*, 2023; Kwon *et al.*, 2023). Store the per-stage table with the eager baseline commit so graph and MegaKernel PRs can diff against it mechanically. A stage table that cannot be regenerated from CI is not a baseline; it is an anecdote that will be re-litigated in every review (Chen and YiRage Contributors, 2026; Chen, 2020, 2026).

1.2 graph mode

Graph mode attacks orchestration, not arithmetic. CUDA Graphs record a static kernel DAG once and replay it with a single host submission, amortizing Python dispatch and driver launch (NVIDIA, 2024; Chen, 2026). The surprise in production is not peak FLOPs regression—it is illegal tier placement: captured graphs still spill activations to HBM when fusion boundaries were never merged, so bytes/token flatlines while launch counts look healthy (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Memory-Floor's controlled A/B on Qwen-2.5-7B (BS=1, ctx=2048) reports 1.259× median speedup on H100 from graph replay alone—roughly 3.05 ms of host overhead removed per step (Chen, 2026). On bandwidth-saturated SKUs the same trick yields only ≈1.03× because R_{floor} is already near the HBM roofline: graphs fix launches, not bytes (Chen, 2026; SemiAnalysis, 2024). Graph mode therefore wins when TaxBreak-style counters show launch-bound decode, not when DRAM bytes dominate (Vellaisamy *et al.*, 2026).

Replayed graphs still execute the same fused or unfused kernel sequence—they do not merge operators (NVIDIA, 2024; Chen, 2026; Chen and YiRage Contributors, 2026). Teams celebrating graph speedups without operator fusion should expect the next plateau when bytes/token remain at eager levels (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026). That plateau is the handoff to MegaKernel mode (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022, 2024; AMD, 2024a).

Operational constraints: capture requires stable addresses, static shapes within a bucket, and no implicit allocations inside the captured region (NVIDIA, 2024; Kwon *et al.*, 2023). Compiler stacks must partition decode into *capture-safe cells*—often one layer block or attention–MLP seam—and re-capture when context length crosses bucket boundaries (Chen and YiRage Contributors, 2026; Chen, 2020). On XDNA, graph analogs are compile-time static schedules, not runtime CUDA capture (AMD, 2024a,b).

Graph mode does not relax memory-bandwidth limits quantified by R_{floor} in Memory-Floor studies: when analytic HBM throughput is already saturated, replay only shaves host time (Chen, 2026; SemiAnalysis, 2024; Davies *et al.*, 2025). Teams should publish both eager and graph traces with launches/token *and* bytes/token side by side—otherwise leadership misreads a 1.25× graph win as permission to skip MegaKernel investment (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023).

Multi-hardware delta. GPU graph mode collapses launches/token; CPU stacks use similar "frozen schedule" ideas via JIT or ORT session plans; NPU fleets require fully static dataflow with DMA edges counted like launches (Goodfellow *et al.*,

2016; AMD, 2024b; Chen, 2020).

Example 1.2.1. If decode is launch-bound on H100 (low R_{floor}, high launches/token), graph replay is the first lever—after verifying KV and weight residency still fit capture legality (Chen, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026).

Graph capture interacts badly with dynamic sampling, speculative decoding branches, and MoE routing unless those paths are lifted outside the captured region (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Kwon *et al.*, 2023). Compiler lowering should emit *graph shells* around stable decoder cores while leaving host-controlled branches eager—mixing modes inside one layer is normal, not a failure (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024). DeepSeek-class deployments still disaggregate prefill and decode pools; graph mode applies per pool, not as a universal whole-model wrapper (Davies *et al.*, 2025; Liu *et al.*, 2024).

On CPU, "graph mode" manifests as frozen execution plans (TorchInductor, ONNX Runtime) with the same trade: amortized dispatch versus static shape buckets (Goodfellow *et al.*, 2016; Chen, 2020). Profile whether replay removed host time or simply hid it in compilation caches before claiming decode wins (Chen, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

CUDA Graph pitfalls include unintended synchronizations inside captured regions, allocator churn from missed caching, and shape drift when KV length crosses bucket boundaries (NVIDIA, 2024; Kwon *et al.*, 2023; Chen, 2026). Operational playbooks bucket context lengths explicitly—for example powers of two—and re-capture graphs per bucket rather than capturing one mega-graph per model (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). When graphs replay but bytes/token unchanged, the next lever is MegaKernel fusion inside the captured DAG, not longer capture windows (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b).

Warmup and capture-time allocation dominate CI stories: first-token latency includes graph recording unless caches are primed (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026). Separate "steady-state decode" metrics from "cold start" when comparing modes—Fregly-style goodput analysis always states which phase is measured (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Kwon *et al.*, 2023). Graph mode wins appear only after warmup when launch storms were the bottleneck (Chen, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b; Goodfellow *et al.*, 2016; Dao

et al., 2022; AMD, 2024a; SemiAnalysis, 2024; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Kwon *et al.*, 2023).

Graph capture lifecycle. Capturing a decode cell is a lifecycle, not a single recording call. The cell first warms up so allocators and kernel variants are stable; the capture pass then records the DAG with concrete buffer addresses; instantiation fixes the memory layout; and steady-state replay submits the same body while input and KV pointers advance between steps (NVIDIA, 2024; Kwon *et al.*, 2023). Each phase has distinct failure signatures: warm-up misses show up as capture-time allocation, steady-state pointer churn shows up as unexpected re-capture, and replay-only allocations abort or silently fall back to eager (Chen, 2026; Davies *et al.*, 2025). Production graph managers should expose the active phase in telemetry so an on-call engineer can tell a genuine replay regression from a bucket-crossing re-capture (Chen and YiRage Contributors, 2026; Chen, 2020). Compiler stacks should emit the phase transitions as IR metadata: which subgraphs are capture-ready, which handles are updated per token, and which regions remain eager by design (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

Bucket sets and context boundaries. One graph per model is a common oversimplification: decode kernels are legal only inside the context bucket they were captured for, because KV block counts, page walks, and shared-memory staging all depend on the range (Kwon *et al.*, 2023; Chen, 2026). Operational stacks should define bucket boundaries explicitly—for example powers of two with a margin—and own a small graph set per bucket rather than re-capturing on every request (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Each set carries a memory cost in resident buffers, so the number of buckets is a fleet-tuning decision that should appear in the mode matrix with its measured hit rate (NVIDIA, 2024; Chen and YiRage Contributors, 2026). When a request crosses a boundary, the fallback path (eager hull or next-bucket graph) must be profiled and logged; silent re-capture inside the serving path is how graph-mode p99 regressions hide (Chen, 2026, 2020).

Honest graph accounting. Graph wins must be measured against the full lifecycle cost. The headline replay number excludes capture wall time, the memory footprint of per-bucket graphs, and first-token latency after a cache miss (NVIDIA, 2024; Chen, 2026). Report three numbers side by side: cold-start ms/token (capture plus replay), steady-state ms/token, and bytes/token unchanged from eager (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). If steady-state improves but

cold start regresses badly, split capture into an offline priming job and keep the serving path on warm graphs (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Graph storage also matters on memory-constrained hosts: dozens of buckets times several model variants can exhaust reserved graph memory, which then thrashes allocators in ways that look like bandwidth problems (Chen, 2026, 2020).

Graph limits across hardware. The graph concept transfers across targets only at the level of *frozen schedules*. CUDA Graphs amortize host dispatch on GPU; ONNX Runtime and JIT plans amortize interpreter overhead on CPU; XDNA/AIE static dataflow removes dispatch entirely at compile time and counts DMA edges as the launch analog (Goodfellow *et al.*, 2016; AMD, 2024b; Chen, 2020). A launch-bound fix on GPU therefore maps to a dispatch-bound fix on CPU and an edge-count fix on NPU, not to the same runtime API (AMD, 2024a; Davies *et al.*, 2025). Triplet regression should compare the lowest legal schedule per backend and annotate which counter each mode moved (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026).

Allocator and fallback semantics. Captured graphs and caching allocators interact through buffer ownership: the capture must either own its buffers or pin them through the graph memory pool, and replay must not trigger an allocation inside the recorded body (NVIDIA, 2024; Chen, 2026). Allocator churn is the most common reason a captured graph silently invalidates or grows memory footprint after a framework upgrade (Kwon *et al.*, 2023; Davies *et al.*, 2025). Define fallback semantics explicitly: if graph replay fails, the serving path should switch to the eager hull, log the failure with the graph bucket and error class, and continue serving instead of crashing or busy-looping on re-capture (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). The fallback rate belongs on the mode dashboard alongside hit rate, because a graph that replays 60% of the time and silently falls back the rest is a p99 regression wearing a success costume (Chen, 2026, 2020).

Fusion staging roadmap. MegaKernel adoption should be staged so each step has a measurable gate. Stage one fuses attention internals (score, online softmax, PV) into one region; stage two carries attention output through the MLP seam; stage three extends the fused envelope across the full layer including residuals and norms; stage four, only if launch cost is still visible, moves toward a persistent kernel (Dao *et al.*, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). Each stage ships only when it beats the previous schedule on both launches/token

and bytes/token at all served context buckets (Vellaisamy *et al.*, 2026; Chen, 2026). Skipping stages conflates failures: a stage-three spill looks identical to a stage-two legality bug unless the byte sheet records which seam was added in which PR (Davies *et al.*, 2025; Chen, 2020). Compiler metadata should track fusion stage as an attribute so rollback can revert one seam without reverting the whole kernel (Chen and YiRage Contributors, 2026; NVIDIA, 2022).

1.3 MegaKernel mode

MegaKernel mode is the decode compiler's endgame: one (or few) device programs per layer that keep QKV, online softmax carriers, MLP activations, and residuals on-chip across seams introduced in Chapters 10–11 (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Graphs remove host tax; MegaKernels remove *HBM round trips* between sub-blocks that eager and graph modes still tolerate (Chen, 2026; Davies *et al.*, 2025).

Legality is the bottleneck. Hopper MegaKernels require shared-memory/TMA budgets that fit fused attention–MLP regions; illegal fusion silently exports tensors and shows up as bytes/token regression versus an unfused baseline (NVIDIA, 2022; SemiAnalysis, 2024; Chen, 2026). XDNA MegaKernels map to static tile buffers and carrier slots with compile-time lifetimes—the same subgraph legal on GPU may be rejected outright on NPU (AMD, 2024b,a; Chen and YiRage Contributors, 2026). CPU MegaKernels appear as cache-blocked super-kernels with NUMA-local KV placement (Goodfellow *et al.*, 2016; Chen, 2020). Register and shared-memory overflow during fusion forces spill to HBM—the failure mode looks like a bandwidth problem in profiles but originates in legality (Chen, 2026; SemiAnalysis, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024).

YiRage encodes residency rules from Chapter 2 as checkable IR metadata so MegaKernel plans fail in the compiler rather than in fleet A/B (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). Manual CUDA teams should treat failed legality as a release blocker: shipping a GPU-only MegaKernel to heterogeneous fleets recreates the framework fragmentation this book removes (Davies *et al.*, 2025; Kwon *et al.*, 2023).

MegaKernel reviews should attach roofline or buffer accounting for fused regions—registers, shared memory, TMA slots, or XDNA tile carriers—before merge, matching the profile-first culture in Chapter 1 (Chen, 2026; SemiAnalysis,

2024; Davies *et al.*, 2025; AMD, 2024a,b; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Fusion that lowers launches but raises bytes is a regression even if nsys screenshots look busy (Vellaisamy *et al.*, 2026; Chen, 2026).

Multi-hardware delta. Identical decoder math can yield 847 launches/token in eager TaxBreak cells versus one replayed graph versus one MegaKernel—but only the MegaKernel path simultaneously cuts bytes/token when fusion keeps KV and activations on-chip (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen, 2026; Chen and YiRage Contributors, 2026).

XDNA MegaKernels compile to AIE dataflow graphs with fixed DMA edges; Hopper MegaKernels compile to CUDA with TMA and DSM (AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020). A fusion plan exported from GPU development must be re-validated on NPU—shared IR does not imply shared legality (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023; AMD, 2024a). MegaKernel mode is the point where cross-hardware compilation pays off or fails loudly (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023).

Example 1.3.1. Decode cell (BS=1, ctx=2048): a legal Hopper MegaKernel fusing attention softmax carriers with the following MLP must show lower bytes/token *and* lower launches/token versus graph replay alone; otherwise fusion did not reach on-chip residency (Dao *et al.*, 2022; Chen, 2026; Chen and YiRage Contributors, 2026).

MegaKernel mode inherits numerics from Chapter 10: online softmax carriers (m, ℓ, o) must survive the KV loop inside the same register/shared-memory budget as the following GEMM (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026). Chapter 11 extends the same carriers across attention–MLP seams; attempting MegaKernel fusion without stable softmax state forces silent HBM spill (Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024). Compiler passes should reject fusion plans that export carriers between subgraphs (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a).

MoE decode adds routing and expert gather/scatter inside the MegaKernel envelope—TaxBreak shows MoE models at 6k–9k launches/token in eager cells, so MegaKernel plans must collapse expert dispatch edges, not only attention (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Kwon *et al.*, 2023). When expert routing stays host-visible, hybrid schedules (graph shell + MegaKernel core) beat pure eager without pretending full-layer fusion is legal (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Blackwell and Hopper DSM/TMA features widen fusion envelopes but do not remove legality checks: double-buffered KV movement still requires planned barriers and shared-memory accounting (NVIDIA, 2022; SemiAnalysis, 2024; Chen, 2026). Manual CUDA MegaKernels should document buffer lifetimes the way YiRage encodes them in IR—otherwise maintenance regressions show up first as silent bytes/token drift, not compile failures (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; AMD, 2024a; Goodfellow *et al.*, 2016).

Persistent MegaKernels—kernels that stay resident across tokens while updating KV pointers—blur the line between graph replay and fusion (NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2026). Treat them as MegaKernel mode with graph-like launch behavior: the win is still bytes/token and carrier residency, not fewer lines of CUDA for its own sake (Dao *et al.*, 2022; Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024b; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; AMD, 2024a; NVIDIA, 2022; Dao *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026). Reviewers should ask for Nsight memory charts on fused regions, not only kernel duration histograms (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024).

Layer-granular versus persistent envelopes. MegaKernel implementations divide into two envelopes with different operating contracts. A layer-granular MegaKernel launches once per decoder layer per token: one device program owns attention through MLP and returns after the layer completes, which keeps residency tight and makes failures easy to localize (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020). A persistent MegaKernel stays resident across tokens, updates KV pointers on the device, and only returns when the batch or context bucket changes (NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026). Persistent envelopes remove the remaining per-token submission cost but

concentrate failure modes: a single illegal buffer update poisons every subsequent token, and debugging requires device-side state dumps rather than kernel launch boundaries (Davies *et al.*, 2025; Chen, 2026). Teams should adopt layer-granular envelopes first, measure bytes/token and launches/token, and escalate to persistent only when launch cost is still visible after fusion (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

MegaKernel legality review checklist. Every fused program should be reviewed against the same checklist used by compiler legality passes: enumerate buffers and their tiers, prove no hot-path allocation, verify the shared-memory or tile-SRAM high-water against the chip model, confirm DMA or TMA edges have producer-consumer ordering, and show that online carriers survive every seam they cross (SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024a; Chen and YiRage Contributors, 2026). The review artifact is a byte sheet plus a control-flow diagram, not a kernel name. If a reviewer cannot find the byte sheet in the PR, the correct response is to ask for it, because the same omission in production will surface as a bytes/token regression attributed to the wrong cause (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Compiler-generated MegaKernels should fail closed when any checklist item cannot be proven; manual CUDA teams should treat the same failure as a merge blocker rather than a TODO (Chen, 2020, 2026).

Determinism and telemetry inside fused programs. Fusion concentrates more arithmetic into one kernel, which raises the stakes for reduction order and debug visibility. Online softmax merges, MoE gathers, and split-KV reductions must document their associativity order because the same inputs can produce bitwise differences across layouts (Dao *et al.*, 2022, 2023; Davies *et al.*, 2025). Deterministic mode should therefore be a compiler flag that locks reduction trees, while interactive serving may accept bounded drift if the tolerance is measured and logged (Chen, 2020; Chen and YiRage Contributors, 2026). Telemetry inside a MegaKernel needs named counters for bytes moved per stage and barrier waits, not only an end-of-kernel timestamp; otherwise the fused program becomes a black box that profiles slower than the eager stack it replaced (Chen, 2026; Vellaisamy *et al.*, 2026; NVIDIA, 2022).

1.4 mode selection

Mode selection is a profile-first decision tree, not a stylistic preference. Start from measured launches/token and bytes/token on the target SKU at the context

Table 1.1: TaxBreak kernel launches per output token (H100 decode, BS=4, SL=2048, m=10) motivate mode escalation (Vellaisamy *et al.*, 2026).

Model	**Launches/token**	**GPU util. (%)**
Llama-3.2-1B	847.5	58.9
Llama-3.2-3B	1,536.9	67.6
OLMoE-1B/7B	9,305.3	15.5
Qwen1.5-MoE-A2.7B	6,695.1	27.7

buckets you ship (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025). If launches dominate and shapes are bucket-static, graph capture is the low-risk win (NVIDIA, 2024; Chen, 2026). If bytes/token dominate with moderate launch counts, invest in MegaKernel legality and compiler metadata (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020). Keep eager PyTorch baselines for debug, dynamic-shape research, and regression diffs—not for production decode pools once counters prove orchestration or residency debt (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

Each mode trades hardware utilization against software flexibility: eager maximizes flexibility at the cost of launches and bytes; graph mode trades dynamic control for launch amortization; MegaKernel mode trades compile-time complexity for residency (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b). Hardware teams should sign off on the trade-off explicitly—for example accepting static context buckets to unlock graph capture on Hopper, or accepting longer compile times to unlock XDNA static schedules (NVIDIA, 2024; AMD, 2024a; Davies *et al.*, 2025; SemiAnalysis, 2024).

High launches/token with moderate utilization (top rows) often respond to CUDA Graph replay first (NVIDIA, 2024; Chen, 2026). MoE rows with extreme launches and low utilization need MegaKernel plus routing collapse—graphs alone cannot fix expert dispatch storms (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025). Always pair launch tables with bytes/token from the same trace; otherwise mode selection optimizes the wrong bottleneck (Chen, 2026; SemiAnalysis, 2024; AMD, 2024b; Chen, 2020; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023).

Cross-hardware regression is mandatory: a mode that wins on H100 launch-bound cells may tie or lose on L4 memory-bound cells or XDNA static schedules (Chen, 2026; AMD, 2024b; SemiAnalysis, 2024). Compiler IR should carry mode annotations so the same decoder lowers to eager, graph, or MegaKernel backends without rewriting model Python (Chen and YiRage Contributors, 2026; Chen,

Table 1.2: Execution mode selection heuristics for batch-1 decode (profile before committing).

Signal	**Prefer**	**Avoid**
High launches/token, static bucket	CUDA Graphs / frozen schedule	Unfused eager
High bytes/token, fusion legal	MegaKernel	Graph-only without fusion
Dynamic shape / debug	Eager baseline	Forced graph capture
NPU fleet	Static MegaKernel / AIE schedule	GPU-only CUDA graphs
Low R_{floor}, high DRAM	MegaKernel + bandwidth opts	Launch-only graph tuning

2020; Goodfellow *et al.*, 2016). Production reviewers should reject mode changes backed only by prefill FLOPs or isolated GEMM microbenchmarks (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Dao *et al.*, 2023).

Example 1.4.1. Rollout checklist: (1) profile eager bytes/token and launches/token at target buckets; (2) if launch-bound, ship graph replay and re-profile; (3) if bytes remain high, iterate MegaKernel legality with compiler metadata; (4) run GPU/CPU/NPU triplet regression before fleet promotion (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; AMD, 2024b; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016).

Fleet operators should log execution mode per deployment cell in telemetry—without that label, postmortems confuse graph wins on launch-bound SKUs with MegaKernel wins on bytes-bound SKUs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; AMD, 2024a; Dao *et al.*, 2022; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Industrial rollout narrative. A typical maturity path starts with Hugging Face eager forwards in research, adds vLLM serving for KV and batching, captures CUDA Graphs on stable decode buckets, then invests in YiRage or hand-written MegaKernels once bytes/token plateaus (Kwon *et al.*, 2023; NVIDIA, 2024; Chen

and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025). Skipping straight to MegaKernels without graph and eager baselines makes regressions unexplainable—teams cannot tell whether fusion failed legality or never beat orchestration (Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Conversely, stopping at graph replay when R_{floor} is already high leaves DRAM-bound decode on the table (Chen, 2026; Vellaisamy *et al.*, 2026). Mode selection is therefore a *sequence* of profile gates, not a one-time architecture slide (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Kwon *et al.*, 2023; AMD, 2024b; Chen, 2020; Goodfellow *et al.*, 2016).

Disaggregated prefill/decode fleets compound the choice: prefill pools may remain eager or graph-captured on wide GEMMs while decode pools push MegaKernels for batch-1 latency (Davies *et al.*, 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Using prefill throughput to justify decode mode leads to systematic under-investment in launch and bytes counters on the decode side—the DeepSeek-style capacity skew exists because phases obey different rooflines (Davies *et al.*, 2025; Dao *et al.*, 2023; Chen, 2026; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; NVIDIA, 2024; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Dao *et al.*, 2022). Compiler teams should expose mode as an IR attribute so backend passes can reject illegal combinations early rather than at link time (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; NVIDIA, 2022; SemiAnalysis, 2024; NVIDIA, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; Liu *et al.*, 2024; Kwon *et al.*, 2023; Dao *et al.*, 2022; AMD, 2024a).

Hardware SLO reviews should include a "mode matrix": rows are SKUs (Hopper, L4, XDNA, CPU), columns are context buckets, cells list chosen mode plus bytes/token and launches/token (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024b,a; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025). Empty cells mean unprofiled deployment—a common source of p99 surprises when fleets autoscale across instance types (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022).

Decision inputs beyond launch and byte counters. Launches/token and bytes/token pick the bottleneck, but deployment picks the mode within the legal set. Compile-time budget, iteration velocity, maintainer skill, and fleet SKU spread all enter the decision: a team shipping one model on one GPU can afford a hand-written MegaKernel, while a fleet that recompiles nightly needs compiler-generated modes with fast legality checks (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). The profile tells you which counter is binding; the operating context tells you which schedule you can sustain across releases (Vellaisamy *et al.*, 2026; Chen, 2026). Record both dimensions in the decision record—otherwise a future reviewer sees only the chosen mode and cannot tell whether graph replay was skipped because bytes were binding or because capture engineering was not funded (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Mode mixing inside a decode step. Real serving stacks rarely run one mode for an entire token. Sampling, speculative branches, MoE routing, and dynamic padding live naturally in eager hulls; stable attention and MLP regions run as captured or fused bodies; persistent kernels may own the KV update loop while per-token scheduling stays host-side (NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024). The compiler's job is to make these seams explicit in the IR and to keep the boundary counters measurable on both sides (Chen and YiRage Contributors, 2026; Chen, 2020). A hybrid schedule that mixes modes per subgraph is a sign of maturity, not a hack, provided each region has a mode attribute and the profile can attribute latency to the right region (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026).

Operational gates and rollback. Mode promotion should follow the same gates as kernel promotion: non-regression on bytes/token, launches/token, and p99 latency at every shipped context bucket; a legalization report for fused regions; and a documented rollback path to the previous mode (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026). Rollback must be a runtime switch, not a redeploy, because fleet incidents will demand switching eager hulls or graph sets without rebuilding binaries (Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2020). Telemetry should tag the active mode and the reason it was chosen (profile-triggered, manual, or fallback), which is the only way postmortems can separate mode regressions from model or data regressions (Kwon *et al.*, 2023; Chen, 2026).

Worked scenario walkthroughs. Consider three fleets. A dense Llama-3.2-class model on launch-bound H100 cells with static buckets starts with graph replay, measures the launch drop, then fuses only if bytes/token remain above the analytic floor (Vellaisamy *et al.*, 2026; Chen, 2026). A MoE model whose eager profile shows expert routing and gather-scatter dominating launches skips attention-only fusion and invests in routing collapse plus a graph shell, because attention fusion alone leaves 6k–9k launches/token untouched (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). An XDNA fleet compiles the same decoder IR to static dataflow from day one; its mode debate is about DMA edge counts and tile-SRAM legality rather than host dispatch, so the graph-mode discussion is replaced by a schedule-legality review (AMD, 2024b,a; Chen and YiRage Contributors, 2026). Each scenario produces a different mode matrix even though the underlying counters are identical—which is exactly why the matrix rows must be SKU and context bucket, not model name alone (Davies *et al.*, 2025; Chen, 2026).

Re-profile cadence and upgrade triggers. Mode decisions rot on a schedule: driver, CUDA, framework, model, context limit, and fleet SKU changes can each move the launch/bytes balance that justified the current mode (NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026). Define explicit triggers that reopen the mode gate—every dependency upgrade above a pinned version, every new context bucket, every model architecture change, and every capacity shift across SKU types (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Between triggers, a lightweight nightly job should compare current counters against the mode matrix and alert when a cell drifts beyond tolerance; the alert is not an automatic mode flip but a reason to run the full profile gate (Chen, 2020; Kwon *et al.*, 2023). This cadence converts mode selection from a one-time architecture slide into a maintained operational contract, which is the only way the escalation path survives team turnover and framework churn (Chen, 2026; Vellaisamy *et al.*, 2026).

1.5 Key Takeaways

- **Three modes, one metric ledger.** Eager, graph, and MegaKernel differ in launches/token *and* bytes/token—optimize the counter your SKU actually bounds (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025).
- **Eager is baseline, not destiny.** Framework dispatch is correct for debug; production decode needs profiled migration to graph or MegaKernel (Kwon *et al.*, 2023; NVIDIA, 2024).

- **Graphs fix host tax.** Memory-Floor reports 1.259× on H100 when launch overhead dominates; graphs do not replace fusion (Chen, 2026; NVIDIA, 2024).
- **MegaKernels fix residency.** Fusion legality on Hopper/XDNA/CPU determines whether bytes/token fall; YiRage-style checks belong in compile time (Chen and YiRage Contributors, 2026; AMD, 2024b; NVIDIA, 2022).
- **Profile-first selection.** Use Table 1.2; never pick mode from prefill FLOPs alone (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026).
- **Multi-hardware parity.** Identical IR must lower per target residency tables—GPU graph capture ≠ NPU static schedule (Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016).

Practitioner mantra. Profile end-to-end decode first. Collapse launches with graphs second. Collapse bytes with MegaKernels third. Re-profile on every bucket, SKU, and framework upgrade (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a). Teach reviewers to reject mode PRs that show only one counter moving (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022). Post the mantra next to decode dashboards so graph and MegaKernel investments stay tied to measured goodput (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a).

1.6 Conclusion

Three kernel execution modes translate Chapter 1 counters into concrete schedules: eager exposes every framework boundary; CUDA Graphs collapse launches when shapes are stable; MegaKernels collapse HBM traffic when fusion is legal (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen, 2026; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). None of the modes substitutes for profiling bytes/token and

launches/token on the SKU and context buckets you operate (Davies *et al.*, 2025; SemiAnalysis, 2024).

Mode choice is reversible across releases: fleets can ship graph replay while MegaKernel plans mature in CI, as long as telemetry tags the active mode and counters prove non-regression (NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022). Treat execution mode as a tunable deployment parameter guarded by profile gates, not as a one-time rewrite (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; NVIDIA, 2024; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a). The escalation path—eager baseline, graph replay, MegaKernel fusion—should appear explicitly in your decode runbooks and compiler IR schemas (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a).

Chapter 13 moves from schedule choice to compiler theory—how IR, passes, and lowering pipelines decide which mode is even emit-able on GPU, CPU, and NPU backends (Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024b; Liu *et al.*, 2024; Kwon *et al.*, 2023; AMD, 2024a; Dao *et al.*, 2023). Carry the mode-selection table forward; later MegaKernel chapters assume you can justify eager vs graph vs fused emission with measured decode goodput, not architecture slides alone (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026).

Readers should leave this chapter able to defend a mode choice in a design review: cite TaxBreak launches/token when proposing graphs; cite Memory-Floor R_{floor} when graphs underperform; cite YiRage legality reports when proposing MegaKernels (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; AMD, 2024a; NVIDIA, 2022). The three modes are not competitors—they are layers in a profile-driven escalation path toward decode goodput (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; NVIDIA, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; SemiAnalysis, 2024; NVIDIA, 2022). Document that escalation in compiler schemas so the next maintainer inherits profile discipline, not folklore (Chen and

YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 19, 20, 21, 22

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 1, 2, 3, 4, 5, 7, 8, 10, 11, 12, 13, 15, 16, 17, 19, 20, 21, 22

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13, 14, 15, 16, 17, 20, 21, 22

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15, 16, 17, 20, 21, 22

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 2, 3, 4, 5, 7, 8, 10, 11, 12, 13, 15, 16, 17, 20, 21, 22

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 15, 16, 17, 18, 19, 20, 21, 22

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 1, 2, 3, 4, 5, 7, 8, 9, 11, 12, 13, 15, 16, 17, 18, 19, 20, 21, 22

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 1, 2, 3, 4, 5, 7, 8, 9, 11, 12, 13, 14, 17, 20, 21, 22

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 15, 16, 17, 18, 19, 20, 21, 22

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 20, 21, 22

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22